MECHATRONICS AND IOT

ASSIGNMENT - 13

Industrial DG Monitoring and Analytics System

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

Industrial DG Monitoring and Analytics System

1. Project Overview

The Industrial DG Monitoring and Analytics System is a prototype designed to monitor important operating parameters of a Diesel Generator (DG) using an ESP8266 NodeMCU. The system measures the electrical current drawn by the load using an ACS712 current sensor and monitors environmental conditions such as temperature and humidity using a DHT11 sensor.

The measured parameters are processed by the ESP8266 and displayed locally on an OLED display. The system provides a simple and low-cost method for observing DG operating conditions and identifying abnormal changes in load and environmental parameters.

The prototype consists of an ESP8266 NodeMCU, ACS712 current sensor, DHT11 temperature and humidity sensor, OLED display, breadboard, and jumper wires.

2. Project Statement

Diesel generators are widely used as backup and prime power sources in industries, commercial buildings, hospitals, and other critical facilities. Continuous monitoring of generator operating conditions is important for efficient operation, maintenance, and early identification of abnormal conditions.

Traditional monitoring systems can be expensive and may require specialized equipment. Therefore, there is a need for a low-cost embedded monitoring system capable of measuring important DG parameters and presenting the information in an easily understandable form.

The proposed project develops a prototype using an ESP8266 NodeMCU to monitor load current, temperature, and humidity and display the readings on an OLED screen.

3. Proposed Solution

The proposed system uses the ESP8266 NodeMCU as the main controller.

The ACS712 current sensor provides an analog signal corresponding to the current flowing through the monitored circuit. Since the ESP8266 NodeMCU has only one analog input, A0 is dedicated to the ACS712.

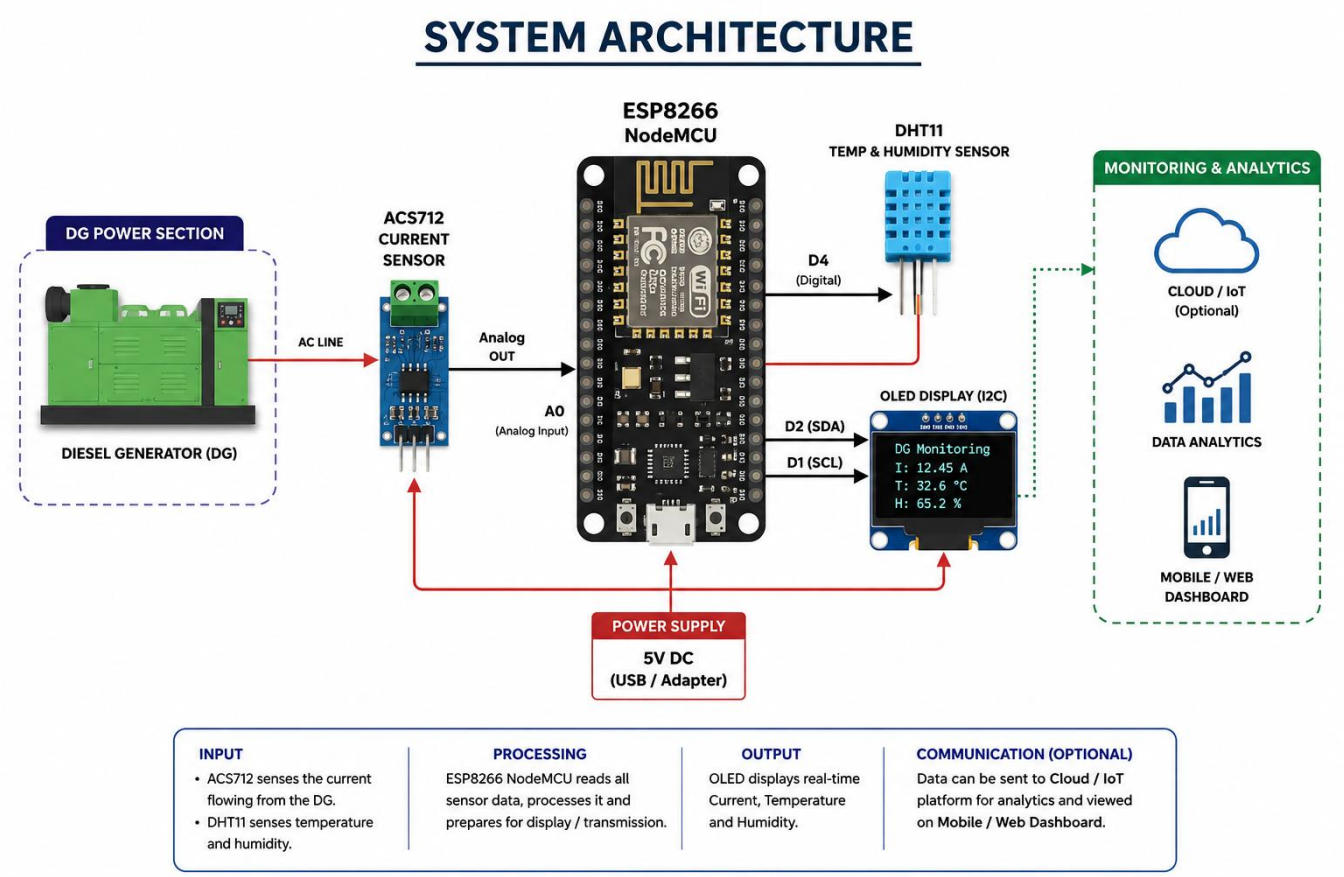
The DHT11 sensor is connected to a digital GPIO pin and provides temperature and humidity measurements. The OLED display communicates with the ESP8266 using the I^2C protocol.

The ESP8266 reads the sensor values, processes the data, and displays the results on the OLED.

Main functions

- Measure DG/load current.
- Measure ambient temperature.
- Measure humidity.
- Process sensor readings using ESP8266.
- Display measurements on OLED.
- Provide serial-monitor output for testing and debugging.
- Provide a foundation for future IoT/cloud-based DG analytics.

4. System Architecture:



5. Circuit Table:

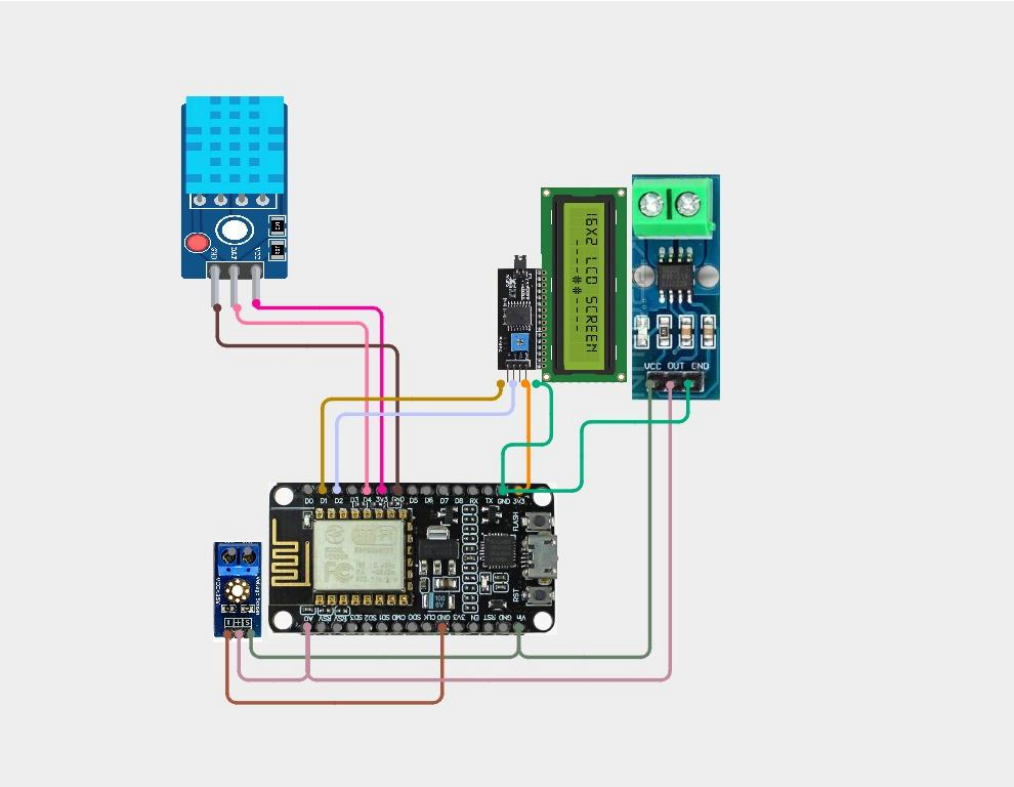
CIRCUIT TABLE

Industrial DG Monitoring and Analytics System

No.	Component	Pin / Terminal	Connected To (ESP8266 NodeMCU)	ESP8266 Pin / Power	Type	Description
1	ACS712 Current Sensor 	VCC	Power Supply (+5V)	VIN / 5V	Power	Powers the current sensor module
		GND	Ground (GND)	GND	Ground	Common ground
		OUT	Analog Output	A0	Analog	Analog output proportional to current
2	DHT11 Temperature & Humidity Sensor 	VCC	3.3V Supply	3.3V	Power	Powers the DHT11 sensor
		GND	Ground (GND)	GND	Ground	Common ground
		DATA	Digital Data Output	D4 (GPIO2)	Digital	Sends temperature & humidity data
3	OLED Display (I2C) 	VCC	3.3V Supply	3.3V	Power	Powers the OLED display
		GND	Ground (GND)	GND	Ground	Common ground
		SDA	I2C Data Line	D2 (GPIO4)	Digital (I2C)	I2C Serial Data
		SCL	I2C Clock Line	D1 (GPIO5)	Digital (I2C)	I2C Serial Clock
4	ESP8266 NodeMCU 	VIN / 5V	Power Supply (+5V)	VIN / 5V	Power	External power input (5V)
		3.3V	3.3V Output	3.3V	Power	Regulated 3.3V output for sensors
		GND	Ground	GND	Ground	Common ground for all components
		A0	Analog Input	A0	Analog	Reads analog signal from ACS712
		D1 (GPIO5)	I2C Clock (SCL)	D1	Digital	I2C Clock Line
		D2 (GPIO4)	I2C Data (SDA)	D2	Digital	I2C Data Line
		D4 (GPIO2)	Digital Input	D4	Digital	Reads data from DHT11
5	Power Supply (USB / Adapter)	+5V	To ESP8266 VIN	VIN / 5V	Power	Provides 5V operating power
		GND	To ESP8266 GND	GND	Ground	Common ground

Note: All GND connections must be common. Ensure the ACS712 supply and output are within the ESP8266 input limits.

6. Circuit Design:



7. Algorithm:

Step-by-step Algorithm

1. Start the system.
2. Initialize the ESP8266 NodeMCU.
3. Initialize the DHT11 sensor.
4. Initialize the OLED display.
5. Initialize the ACS712 current sensor input.
6. Calibrate the ACS712 zero-current offset.
7. Read the analog signal from the ACS712.
8. Convert the analog value into current in amperes.
9. Read temperature from the DHT11.
10. Read humidity from the DHT11.

11. Check whether the sensor readings are valid.
12. Display current, temperature, and humidity on the OLED.
13. Send the readings to the Serial Monitor.
14. Repeat the measurement process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
// ================================================
// Wi-Fi DETAILS
// ================================================
#define WIFI_SSID       "OnePlus Nord CE5 7C5C"
#define WIFI_PASS       "qkts5737"
// ================================================
// ADAFRUIT IO DETAILS
// ================================================
#define IO_USERNAME     "Sivasaravanan"
#define IO_KEY          "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
// ================================================
// PINS
// ================================================
#define ACS_PIN A0
#define DHT_PIN D4
#define DHT_TYPE DHT11
// ================================================
// OLED
```

```
// ================================================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(  SCREEN_WIDTH,SCREEN_HEIGHT,&Wire, OLED_RESET);
// ================================================
// DHT
// ================================================
DHT dht(DHT_PIN, DHT_TYPE);
// ================================================
// ADAFRUIT IO FEEDS
// ================================================
AdafruitIO_Feed *currentFeed =io.feed("current");
AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
AdafruitIO_Feed *humidityFeed = io.feed("humidity");
// ================================================
// ACS712 SETTINGS
// ================================================
// For ACS712 20A
// 20A = 100 mV/A
const float ACS_SENSITIVITY = 0.100;
// ESP8266 ADC
const int ADC_MAX = 1023;
const float ADC_VOLTAGE = 3.3;
// Zero-current offset
float zeroCurrentVoltage = 1.65;
// ================================================
// SETUP
// ================================================
void setup()
{
```

```
Serial.begin(115200);
delay(1000);
// ---------------- DHT ----------------
dht.begin();
// ---------------- OLED ----------------
Wire.begin(D2, D1);
if (!display.begin(SSD1306_SWITCHCAPVCC,0x3C))
{
  Serial.println("OLED not found!");
  while (1);
}
display.clearDisplay();
display.setTextColor(SSD1306_WHITE);
display.setTextSize(1);
display.setCursor(20, 10);
display.println("INDUSTRIAL DG");
display.setCursor(25, 25);
display.println("MONITORING");
display.setCursor(35, 40);
display.println("SYSTEM");
display.display();
delay(2000);
// ---------------- ACS712 CALIBRATION ----------------
calibrateACS712();
// ---------------- CONNECT TO ADAFRUIT IO ----------------
display.clearDisplay()
display.setCursor(0, 20);
display.setTextSize(1);
display.println("Connecting to");
display.println("Adafruit IO...");
display.display();
```

```
  Serial.println();
  Serial.println("Connecting to Adafruit IO...");
  io.connect();
  // Wait for connection
  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.println("Connected to Adafruit IO!");
  display.clearDisplay();
  display.setCursor(0, 20);
  display.println("Adafruit IO");
  display.println("Connected!");
  display.display();
  delay(2000);
}
// ================================================
// ACS712 CALIBRATION
// ================================================
void calibrateACS712()
{
  long total = 0;
  Serial.println( "Calibrating ACS712..." );
  Serial.println( "Make sure NO CURRENT is flowing.");
 for (int i = 0; i < 500; i++)
  {
    total += analogRead(ACS_PIN);
   delay(2);
  }
```

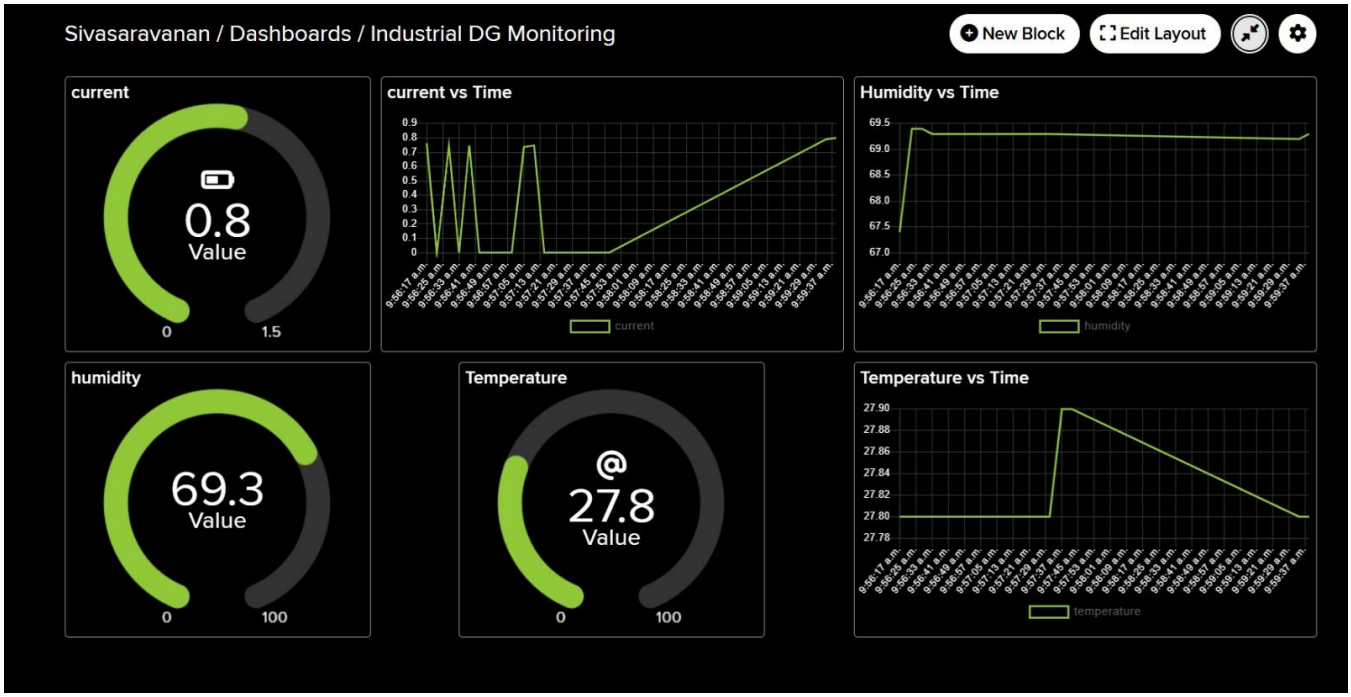
```
  float averageADC =total / 500.0;
  zeroCurrentVoltage = (averageADC / ADC_MAX) *ADC_VOLTAGESerial.print("Zero current voltage: ");
  Serial.println( zeroCurrentVoltage, 3 );
}
// ====================================================
// READ CURRENT
// ====================================================
float readCurrent()
{
  const int samples = 200;
  float sum = 0;
  for (int i = 0; i < samples; i++)
  {
    int adcValue =analogRead(ACS_PIN);
    float voltage =(adcValue /(float)ADC_MAX) *ADC_VOLTAGE;
    float difference =voltage - zeroCurrentVoltage;
    float current =  difference / ACS_SENSITIVITY;
    sum += current;
    delayMicroseconds(500);
  }
  float current =sum / samples;
  // Remove small noise
  if (abs(current) < 0.10)
  {
    current = 0;
  }
  return abs(current);
}
// ====================================================
// MAIN LOOP
// ====================================================
```

```
void loop()
{
  // Keep Adafruit IO connection alive io.run();
  // ================================================
  // READ SENSORS
  // ================================================
  float current =readCurrent();
  float temperature =dht.readTemperature();
  float humidity = dht.readHumidity();
  // ================================================
  // CHECK DHT
  // ================================================
  if (isnan(temperature) ||isnan(humidity))
  {
   Serial.println("DHT11 reading failed!");
   return;
  }
  // ================================================
  // SERIAL MONITOR
  // ================================================
  Serial.println();
  Serial.println( "----------------------------" );
  Serial.print("Current: ");
  Serial.print(current,2);
  Serial.println( " A");
  Serial.print( "Temperature: " );
  Serial.print( temperature,1);
  Serial.println(" C" );
  Serial.print("Humidity: " );
  Serial.print(humidity, 1 );
  Serial.println(" %");
```

```
// ================================================
// SEND DATA TO ADAFRUIT IO
// ================================================
Serial.println("Sending data to Adafruit IO...");
currentFeed->save(current);
temperatureFeed->save(temperature);
humidityFeed->save(humidity);
Serial.println( "Data sent successfully!" );
// ================================================
// OLED DISPLAY
// ================================================
display.clearDisplay();
display.setTextColor(SSD1306_WHITE);
// Title
display.setTextSize(1);
display.setCursor(0, 0);
display.println("DG MONITORING" );
display.drawLine( 0,10,127,10,SSD1306_WHITE );
// Current
display.setCursor(0, 16);
display.print("Current:  ”);
display.print( current, 1 );
display.println( " A");
// Temperature
display.setCursor(0, 32);
display.print("Temp: " );
display.print( temperature, 1 );
display.println(" C" );
// Humidity
display.setCursor(0, 48);
display.print("Humidity: ");
```

```
  display.print(humidity,   0 );
  display.println(" %" );
  display.display();
  // ================================================
  // WAIT
  // ================================================
  delay(5000);
}
```

9. System Working :



When the system is powered ON, the ESP8266 initializes the ACS712, DHT11, and OLED display.

Initially, the system performs a zero-current calibration of the ACS712. During this calibration, no current should be flowing through the sensor. The obtained zero-current value is used as the reference for current calculation.

During normal operation, current flowing through the ACS712 produces a corresponding analog output voltage. The ESP8266 reads this voltage through its A0 analog input and calculates the current value.

At the same time, the DHT11 measures the surrounding temperature and humidity. The ESP8266 receives these measurements through D4.

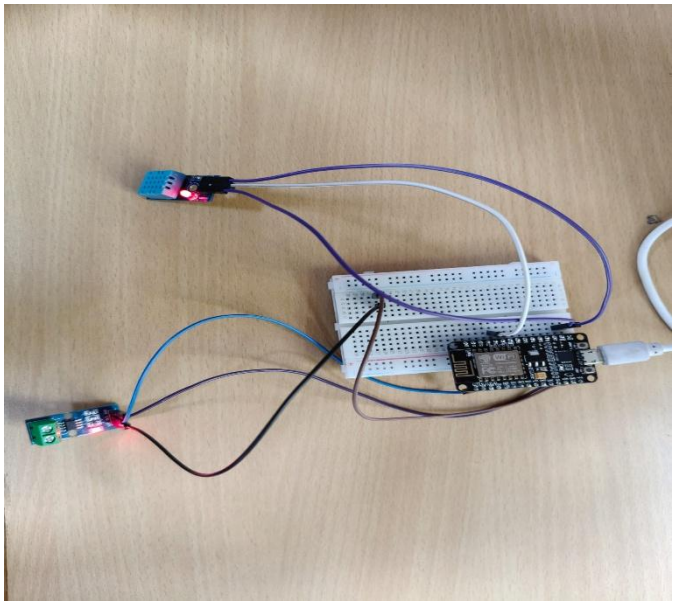
The processed readings are then displayed on the OLED. The same values are also transmitted through the Serial Monitor for testing, recording, and analysis.

The complete operation is repeated continuously, allowing the user to observe changes in the DG operating conditions.

10. BILL OF MATERIAL:

S. No.	Component	Quantity	Description / Specification
1.	ESP8266 NodeMCU	1	Wi-Fi enabled microcontroller board for data acquisition and transmission
2.	ACS712 Current Sensor	1	Hall-effect based current sensor (5A/20A/30A as per requirement)
3.	DHT11 Temperature and Humidity Sensor	1	Measures ambient temperature and relative humidity
4.	0.96-inch I^2C OLED Display	1	OLED display with I^2C interface, 128x64 resolution
5.	Breadboard	1	Solderless breadboard for prototyping connections
6.	Jumper Wires	As required	Male-to-male / Male-to-female jumper wires
7.	USB Cable	1	Micro USB cable for power supply and programming
8.	5 V USB Power Supply	1	5 V DC output power adapter (1A or higher recommended)
9.	Computer / Laptop for Programming and Monitoring	1	For programming NodeMCU and monitoring sensor data

11. Prototype Implementation:



The prototype was implemented by mounting the ESP8266 NodeMCU, ACS712, DHT11, and OLED display on a breadboard.

The ACS712 output was connected to the A0 analog input of the ESP8266 because the NodeMCU provides only one analog input. The DHT11 was connected to D4, while the OLED was connected through the I^2C interface using D1 and D2.

After uploading the program, the system was powered through the USB connection. The OLED displayed the project title initially and then displayed current, temperature, and humidity readings.

The ACS712 was calibrated before measurement. The current measurement was tested by applying different loads to the monitored circuit and observing the corresponding current readings.

For prototype testing, low-voltage and properly isolated test arrangements should be preferred. Direct connection of hazardous generator or mains wiring to a breadboard is not recommended.

12. Results and Performance Analysis:

The developed prototype successfully demonstrated real-time monitoring of generator-related electrical and environmental parameters.

The ACS712 provided current measurements that were processed by the ESP8266 and displayed on the OLED. When the electrical load was changed, the current reading changed accordingly, demonstrating the ability of the system to monitor load variations.

The DHT11 successfully measured temperature and humidity. These values were continuously updated and displayed along with the current measurement.

The OLED provided a convenient local interface because the operator could observe the parameters without requiring a computer. The Serial Monitor additionally provided numerical readings useful for testing and debugging.

The system performed continuously during prototype testing and demonstrated the basic functionality required for a low-cost DG monitoring system.

However, the prototype has some limitations. The ACS712 is an analog current sensor and may require calibration for accurate measurements. Its accuracy can also be affected by electrical noise and the selected sensor range. The DHT11 provides relatively slow and basic environmental measurements. Furthermore, the present prototype does not measure generator voltage because the ESP8266 has only one analog input and an additional ADC or multiplexer was not used.

Future improvements can include:

- Voltage monitoring using an isolated voltage sensor and external ADC.
- Power and energy calculation.
- IoT/cloud data logging.
- Mobile application monitoring.
- Automatic fault alerts.
- Over-current and over-temperature alarms.
- Historical data analysis.
- Predictive maintenance using collected DG operating data.

Overall, the prototype demonstrates that an ESP8266-based system can provide a simple, economical, and expandable platform for monitoring important DG operating parameters.